USB 2.0 Verification Plan

Revision 0.1

USB 2.0 Documentation

AUTHORS

Shruti Patel

Sneha patil

Chandan B V

Pavan Kumar P

Table of contents

Table of Figures

Table of Tables

Revision History

Version	Date	Modified by	Changes
0.1	24/12/2025	Shruthi Patel, Sneha Patil, Chandan, Pavan Kumar P	Initial Changes

1.0 INTRODUCTION

Universal Serial Bus an industry standards that defines the cables, connectors, and communication protocols used for connection, communication, and power supply between computers and electronic devices. The USB communication is always initiated by a Host and responded by a Device.To simplify and standardize the connection of peripheral devices to computers and other hosts.
This document will provide you the information and reference documents. The verification process and given the brief introduction on the USB Verification plan and TB operations of the USB 2.0.
It provides the scope of UVM Testbench Architecture of USB 2.0. It includes functional coverage and assertions along with test scenarios.

USB 2.0 OVERVIEW

USB – Universal Serial Bus. First designed to allow connections to the PC without expansion cards , the USB became a communication standard for almost all electronic devices.

1.2 Earlier Versions of USB:

- USB 1.0: Specified data rates of 1.5Mbit/s (Low Bandwidth)
- USB 1.1: Specified data rates of 12Mbits/s.
- USB 2.0: Added high maximum bandwidth of 480Mbits/s.
- USB 3.0/3.1: Having higher data rates of 5GBit/s called as Super speed.

1.3 Different Speeds for USB 2.0:

- Low Speed: 1.5Mbits/s.
- Full Speed: 12Mbits/s.
- High Speed: 480Mbits/s.

1.4 USB Connectors

- USB Type-A: The classic rectangular connector (typically on host computers/chargers)
- USB Type-B: Square-ish often on printers and scanners.
- Micro-USB: Small connector, common on older smartphones and accessories.
- USB Type-C: The modern standard reversible plug, versatile, supports high power and data.

1.2 USB Applications

List of examples of USB connects: Data storage (Flash drives, external hard drives), Human Interface Devices (Keyboard, Mouse, Game controllers), Imaging (Webcams, Scanners, Printers), Charging (Smartphones, Tablets, Laptops), Audio/Video (Headsets).

1.3 Features of USB:

- Plug and Play: Devices are automatically detected and configured by the operating systems.
- Hot Plugging: Devices can be connected or disconnected while the computer is running without rebooting.
- High Speed: Supports fast data transfer rates.
- Single Standard: One type of interfaces for many devices (keyboards, mice, printers, cameras, etc...)
- Power and Data in one cable: The host gives 5V power and also transfers data through the same cable.
- Host-controlled: Devices cannot talk to each other directly, everything goes through the host.

USB system is described by three definitional areas:

- USB Host
- USB Devices
- USB interconnect

3.0 USB HOST:

- There is only one host in USB system.
- Host controls all communication on the USB bus.
- It decides which device communicates, when it does, and how power is distributed.
- The host supplies power to the device and requests data as needed.

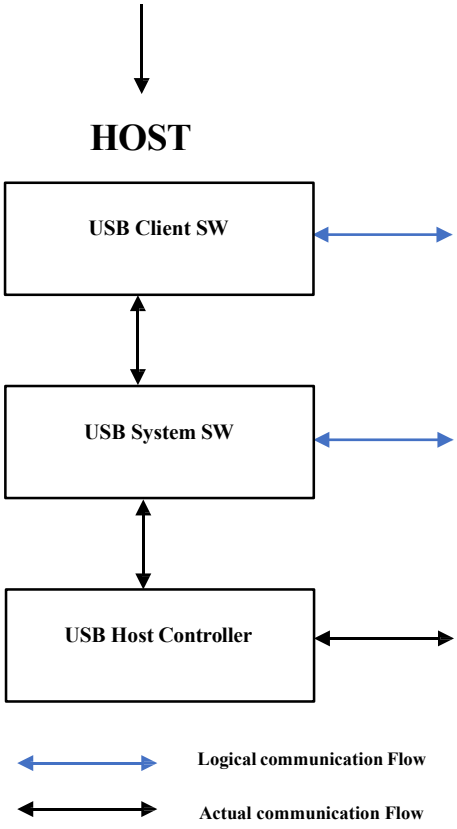


Fig No 1.0 Host Composition

The host's logical composition includes:

- USB Client Software
- USB System Software
- USB Host Controller

3.1 USB Client Software:

- USB client software runs on the host system. Interfaces with USB System Software to communicate with a specific USB Device.
- This is the application uses a USB device.
- It doesn't talk directly to hardware, instead it make requests through higher-level APIs provided by operating system.

3.2 USB System Software:

- USB system software acts as a bridge between client software and a hardware.
- It includes the OS's USB stack, device drivers and class drivers. It manages enumeration (detecting and configuring devices when plugged in).
- Loads the correct driver based on device descriptors.
- Implements Control, Bulk, Interrupt and Isochronous transfers.

3.3 USB Host Controller:

- The USB Host controller is responsible for managing communication between host and the USB Device.
- It is the hardware circuit inside the host that physically controls USB signalling.
- It sends tokens, schedules transfers.
- Works with system software via a Host Controller Driver (HCD).

4.0 USB DEVICE:

- USB Device is any peripheral/ piece of hardware that connects to a host using USB cables and connectors.
- Each device provides structured information (device descriptors, configuration descriptor, interface descriptors, endpoint descriptors) so the host knows what the device is and how to use it.
- Devices can draw power from the USB port.

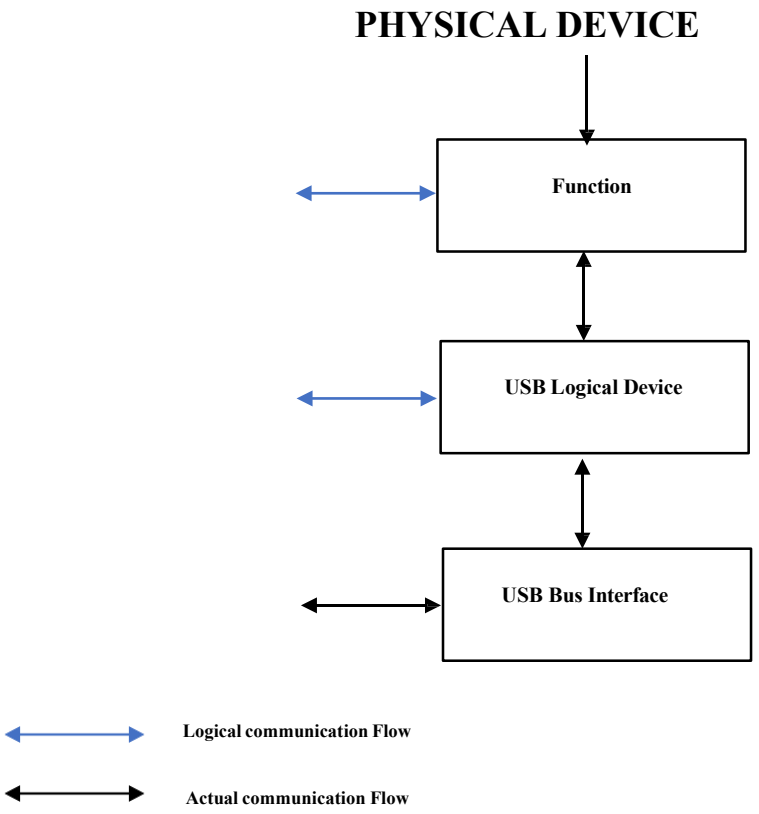


Fig No 1.1 Physical Device Composition

The Device's logical composition includes:

- USB Bus interface
- USB logical device
- Function

4.1 USB Bus interface:

- USB bus interface is the lowest layer of the device that connects physical and electrical connection between the device and the USB host.
- It handles signalling (D+ and D- differential lines).
- It manages packet framing, bit stuffing, sync patterns, CRC checks.
- It provides power interface (drawing current from VBUS).

4.2 USB Logical Device:

- USB logical device defines device descriptors (Vendor ID, product ID, class) and provides configuration, interface, and endpoint descriptors.
- Manages addressing, each device gets a unique address during enumeration.
- Implements control endpoint (endpoint 0) for setup and configuration requests.

4.3 USB Function:

- Function is the actual operational logic of the USB device.
- Each function is implemented using one or more endpoints.
- It handles class-specific or vendor-specific requests.
- It manages data flow through bulk, interrupt or isochronous endpoints.

5.0 USB BUS TOPOLOGY

The USB connects USB devices with the USB host. The USB physical interconnect is a tiered star topology. A hub is at the centre of each star. Each wire segment is a point-to-point connection between the host and a hub or function, or a hub connected to another hub or function.

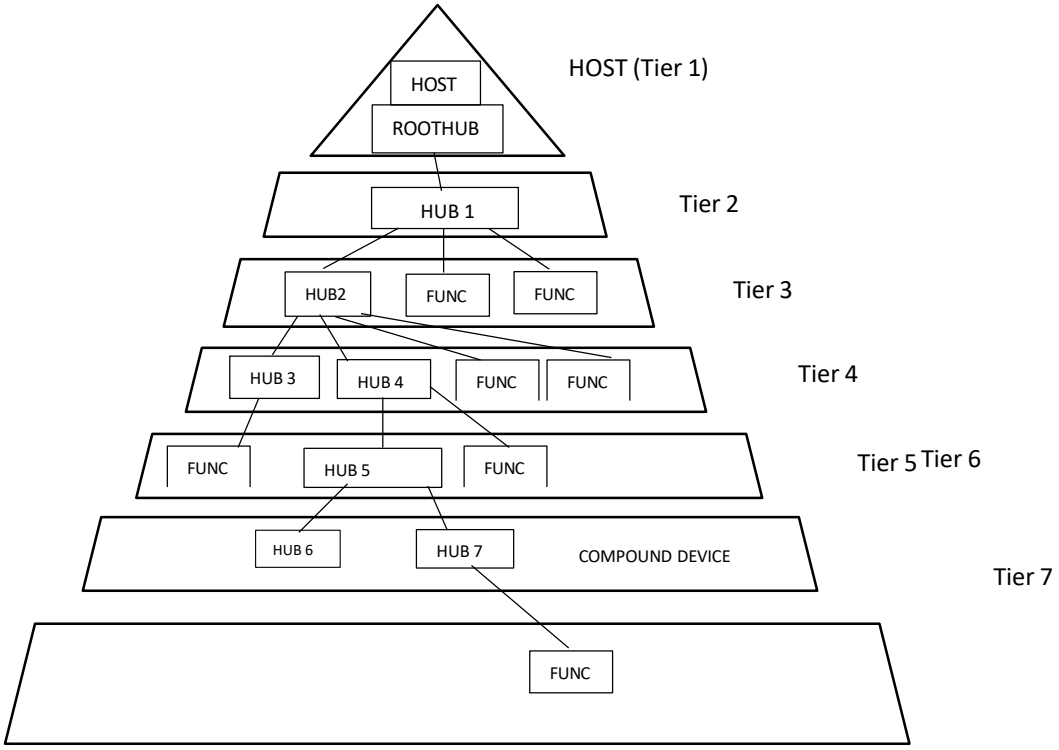
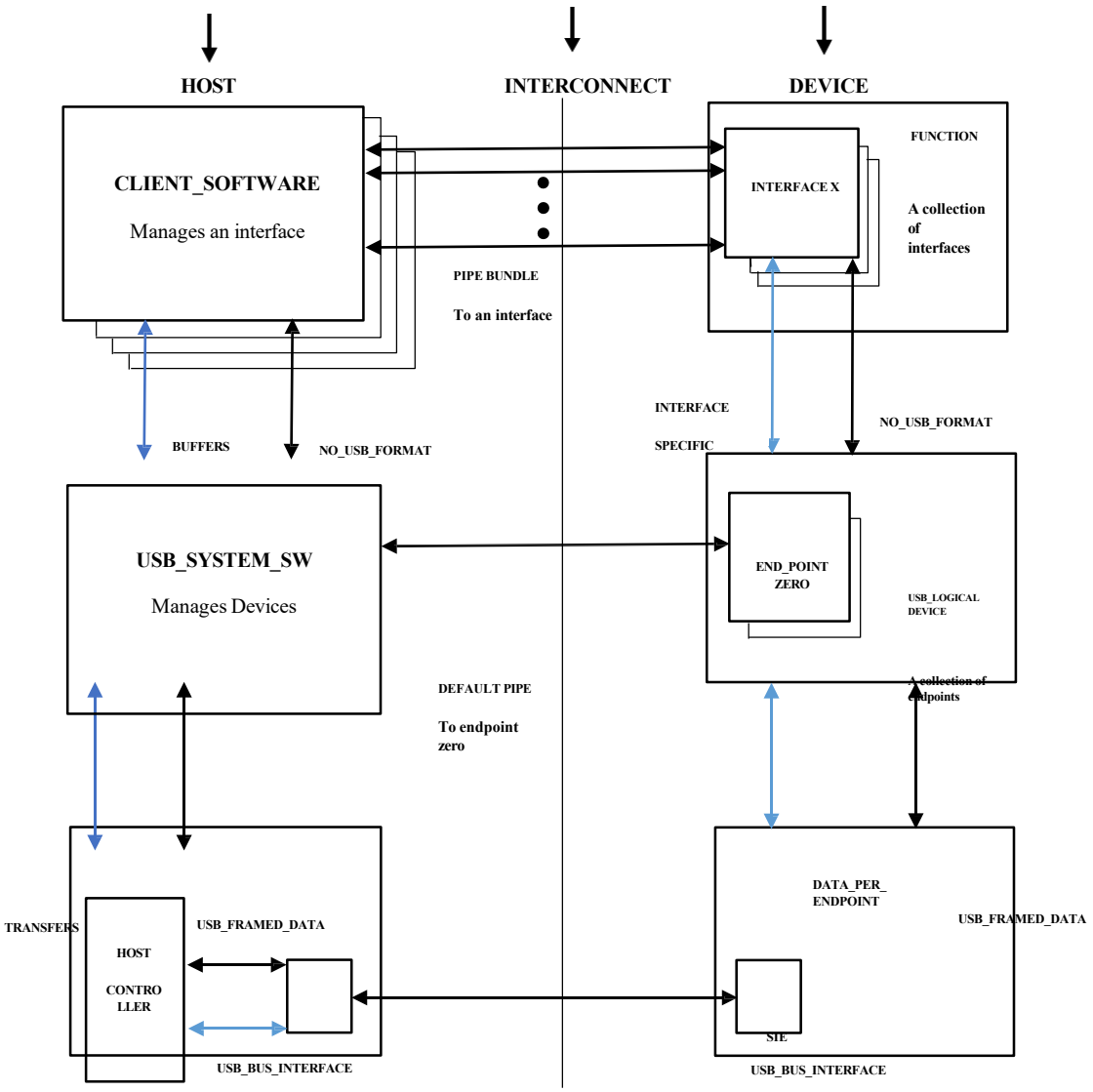


Fig No 1.2 USB Topology

Due to timing constraints allowed for hub and cable propagation times, the maximum number of tiers allowed is seven (including the root tier). Note that in seven tiers, five non-root hubs maximum can be supported in a communication path between the host and any. device. A compound device occupies two tiers therefore, it cannot be enabled if attached at tier level seven. Only functions can be enabled in tier seven.

6.0 USB COMMUNICATION FLOW

The USB provides a communication service between software on the host and its USB function. Functions can have different communication flow requirements for different client-to-function interactions. The USB provides better overall bus utilization by allowing the separation of the different communication flows to a USB function. Each communication flow makes use of some bus access to accomplish communication between client and function.
Each communication flow is terminated at an endpoint on a device. Device endpoints are used to identify aspects of each communication flow.



7.0 USB ARCHITECTURE

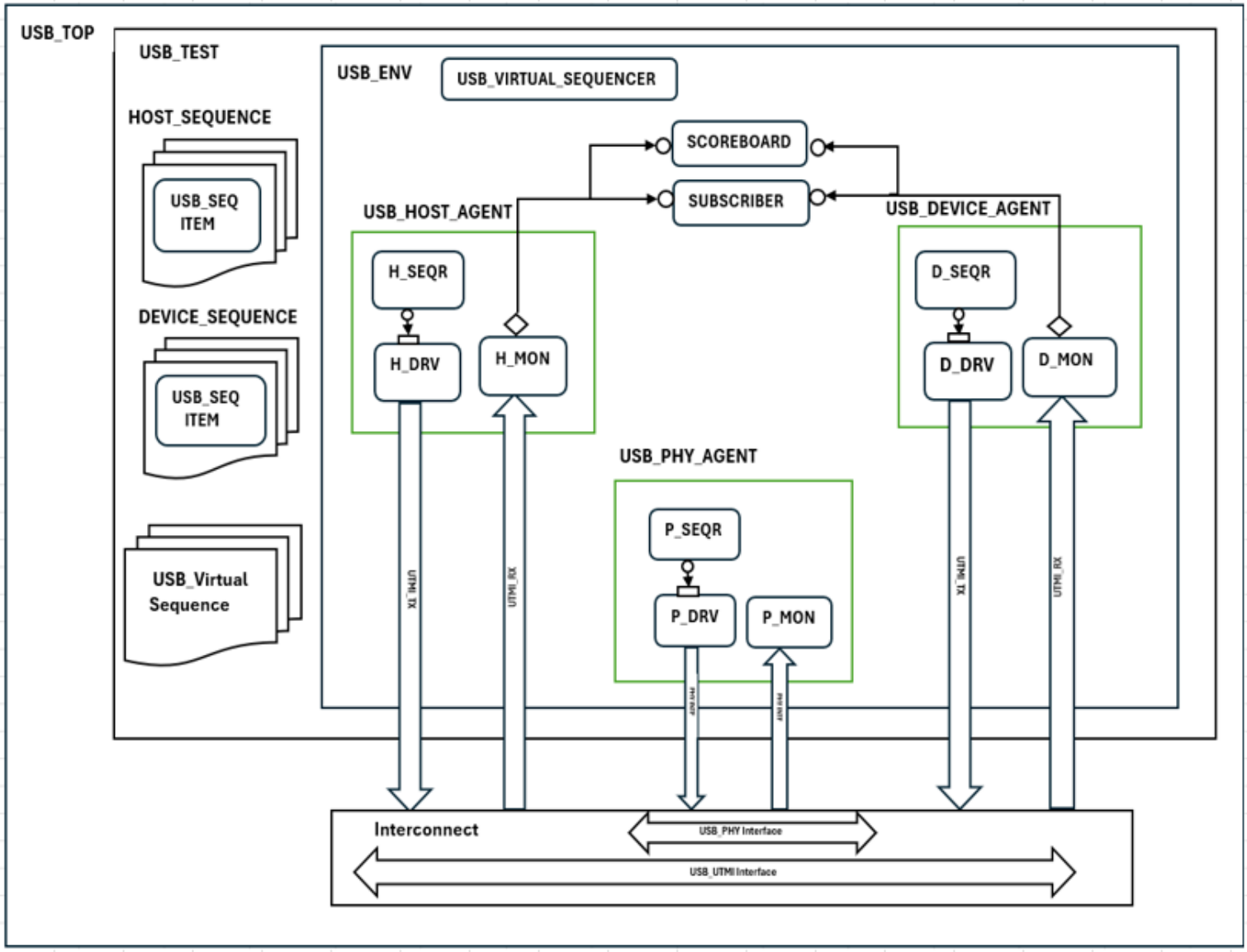


Fig.No 7.1usb architecture

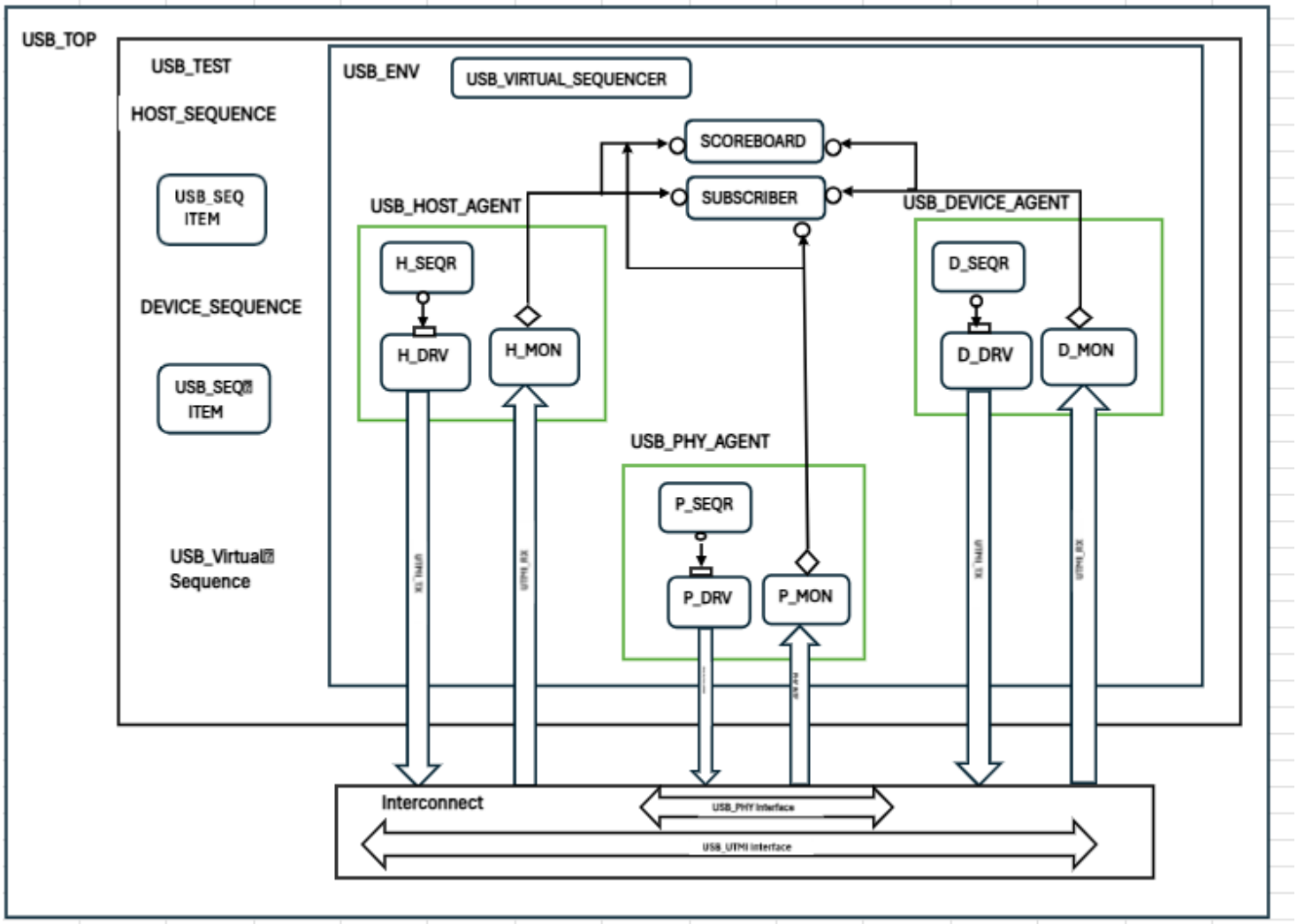


Fig.No 7.2 **usb architecture**

8.0 TEST BENCH DESCRIPTION

8.1 TOP MODULE: Top module is a static container class that has an instantiation of Interfaces. These interface instantiation will connect interface clock with Top module clock. The clock is generated in the top module based on the required frequency for different interfaces. The interface will be stored in the config db by using set method and it will be retrieved in down hierarchies by using get method. And we have done interconnections of interface signals for transmitting the data. The Tb top will be triggering the test by using run_test() method. Test build and simulation will be started once tb triggered the run_test().

8.2 TEST: The test is the topmost class component, using the tests we will be able to cover different functional scenarios of the design for verification. It is responsible for constructing the environment which is in next level hierarchies of the test. In test we will start the virtual sequence which has sequences of transmitters and receivers.

8.3 ENVIRONMENT: The environment is a container class which contains agent and a scoreboard. Environment class is derived from uvm_environment, uvm_environment is inherited from uvm_component. The UVM components scoreboard and agent are built here in the build phase of the environment. The monitor is connected to Scoreboard using analysis ports in the connect phase of the environment.

8.4 HOST-CONTROLLER-AGENT: This Agent consists of Sequencer, Driver and Monitor. These components will take care of driving and receiving of data through interface.

8.5 DEVICE-CONTROLLER-AGENT: This Agent consists of Sequencer, Driver and Monitor. These components will take care of driving and receiving of data through interface.

8.6 PHY-AGENT: This Agent consists of Sequencer, Driver and Monitor. These components will take care of driving and receiving of data through interface.

8.8 DRIVER: Driver will be generally responsible for driving the packed level data to pin level signal. We will declare the virtual interface. Get the interface handle from config_db. Add driver logic in the run phase, get the sequence_item from sequencer and drives to DUT signals. Driver uses TLM port (seq_item_port) to receive transaction items from sequencer and use interface to drive DUT signals. Driver is responsible for sending the write and read packets to the interface through the DUT.

8.9 MONITOR: Monitor will be generally collecting the pin level signals and send it in packet data to scoreboard, and subscriber. Declare virtual interface. Connect interface to virtual interface by using get method. Declare analysis port, declare sequence_item handle which is used to capture the transaction information, send the transaction data to the

scoreboard. Monitor will capture the sending data as well as receives the data and sends the both data packets to scoreboard through analysis port. And it will send only received data packet to subscriber and corresponded driver through analysis port.

8.7 HOST-SEQUENCE: Defines the sequence in which the data items need to be generated and sent to the driver. Sequence class is derived from uvm_sequence class which is parameterized with sequence item, here the parameter will be Seq_item.

8.10 SUBSCRIBER:

Subscriber is extended from uvm_subscriber. The subscriber receives transactions from the monitor via analysis port and collect functional coverage based on content of the transaction. There will be one write method to receive transaction from the monitor. Create Covergroups for Functional Coverage. Samples the value of cover points when data is written by the analysis port.

9.0 TYPES OF TRANSFERS

1. **Control Transfer**
2. **Bulk Transfer**
3. **Interrupt Transfer**
4. **Isochronous Transfer**

9.1Control Transfer:

- Control transfer are intended to support configuration command status type communication flows between client and its function.
- The Default Control Pipe provides access to the USB device's configuration, status, and control information. Always goes through endpoint 0 (Control Endpoint).Supports IN and OUT directions.

Example: Device enumeration, setting device address.

9.2 Bulk Transfers:

- Bulk data typically consists of larger amounts of data.

Example: printers or scanners

5.1 Interrupt Transfer:

- A limited-latency transfer to or from a device is referred to as interrupt data.

Example: Keyboard or Mouse

5.2 Isochronous Transfer:

- Used for continuous data streams requiring constant bandwidth. Example: Audio or video streaming from USB device.

10.0 ENUMERATION

10.0 USB Enumeration

- Process of detecting and configuring a USB device after connection.
- USB allows USB device to attach
- At this point USB device is in powered state and the port to which it is attached.
- The host issues reset command to the port, device enters default state.
- Host assigns unique address to the USB device and moving the device address state.
- Host reads device descriptor to determine max packet size.

10.1 Types of USB Descriptors

- Device Descriptor – General info (VID, PID, USB version, max packet size).
- Configuration Descriptor – Power requirements, it can support number of interfaces.
- Interface Descriptor – Defines functionality(class/subclass)
- Endpoint Descriptor – Endpoint address, type (Control, Bulk, Interrupt, Isochronous), max packet size.

11.0 USB PACKET TYPES

1 Token Packet

2 Data Packet

3 Handshake Packet

11.1 TOKEN PACKETS

There are three types of token packets:

- In
- Out
- Setup – used to begin control transfer

11.1.1 TOKEN PACKET FORMAT

SYNC 8/32 BIT	PID 8BIT	ADDR 7BIT	ENDP 4BIT	CRC 5BIT	EOP

11.2 DATA PACKETS

- There are two types of data packets each capable of transmitting up to 1024 bytes of data
- DATA0
- DATA1
- High speed mode defines another two data PIDs, DATA2 and MDATA.

11.2.1 DATA PACKET FORMAT

SYNC 8/32 BIT	PID 8BIT	DATA 8/1023/ 1024 BYTE	CRC 16BIT	EOP

11.0 HAND SHAKE PACKET

- There are 3 types of hand packets which consist of the PID
  - ACK
  - NAK
  - STALL

11.0.1 HAND SHAKE FPACKET FORMAT

SYNC 8/32 BIT	PID 8BIT	EOP

12.0 CONTROL TRANSFER

- Control transfer are typically used for command and status operations.
- They are essential to setup a USB device with all enumeration functions performed using control transfers.

The packet length of control transfers

- Low speed devices must be 8 bytes
- Full speed devices must be 8,16,32 and 64 bytes
- High speed devices must be 64 bytes

There are 3 stages

- Setup Stage
- Data Stage
- Status Stage

12.1 SET UP STAGE

When the request is sent, each stage consists of 3 packets

- Setup token
- Data packet (8 bytes)
- Handshake packet (ACK)

Setup transaction:

SETUP TOKEN	DATA0 (8BYTES) DATA	ACK HAND SHAKE

EXAMPLE

- Bmrequest - 1000 0000b
- Brequest – 06 - GET_DESCRIPTOR
- wvalue – (parameter value of device, configuration, endpoint,string,interface)
- windex – 01 - English – (language ID)
- Wlength – 0 – out, 1 – in (Direction 1)

12.2 DATA STAGE

- Optional, consists of one or multiple IN/OUT transfers
- Setup request indicates amount of data to be transmitted. Multiple data packets possible
- IN and OUT transfer.

Data transaction:

IN | OUT TOKEN	DATA10 DATA	ACK NACK STALL HAND SHAKE

12.0 STATUS STAGE

- Reports the status of overall request and once varies due direction of transfer.
- Status reporting is always performed by the function.

Status transaction:

IN | OUT TOKEN	DATA1 DATA (0BYTE)	ACK NACK STALL HAND SHAKE

13.0 BULK TRANSFER

- Bulk transfer defines maximum payload size support

- HIGH SPEED – 8,16,32,64 Bytes
- FULL SPEED – 512 Bytes
- LOW SPEED – Bulk endpoints are not supported

TYPES OF TRANSACTION

1. IN – TRANSACTION
2. OUT – TRANSACTION

13.1 IN – TRANSACTION

Host used to send a request data from bulk IN endpoint

- **HOST**– IN TOKEN
- **DEVICE** –DATA0 / DATA1
- **HOST** - ACK

INTOKE	DATA	ACK

13.2 OUT – TRANSACTION

Host sends data to bulk OUT endpoint

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1
- **DEVICE** - ACK

OUTTOKEN	DATA	ACK

14.0 INTERRUPT TRANSFER

Interrupt transfer defines maximum payload size support

- LOW SPEED – 8 Bytes
- FULL SPEED – 64 Bytes
- HIGH SPEED - 1024 Bytes

TYPES OF TRANSACTION

1. **IN – TRANSACTION**
2. **OUT – TRANSACTION**

14.1 IN – TRANSACTION

- **HOST** – IN TOKEN
- **DEVICE** – DATA0 / DATA1
- **HOST** - ACK

INTOKEN	DATA	ACK

14.0 OUT – TRANSACTION

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1
- **DEVICE** – ACK

OUTTOKEN	DATA	ACK

15.0 ISOCHRONOUS TRANSFER

Isochronous transfer defines maximum payload size support

- LOW SPEED – Not supported
- FULL SPEED – 1023 Bytes
- HIGH SPEED - 1024 Bytes

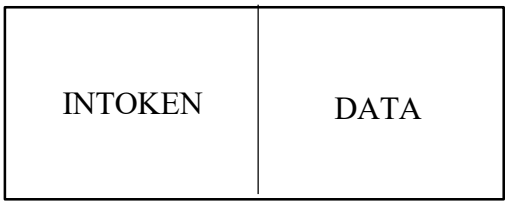
TYPES OF TRANSACTION

1 IN – TRANSACTION

2 OUT – TRANSACTION

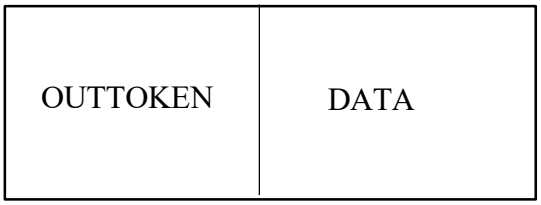
15.1 IN – TRANSACTION

- **HOST** – IN TOKEN
- **DEVICE** – DATA0 / DATA1



15.2 OUT – TRANSACTION

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1



16.0 Assertions:

Sl no	Assertion name	Assertion Property
1	usb2p0_dp_dm_comp	property p_dp_dm_complement; @(posedge phy_clk) (Dp !== Dm); endproperty
2	usb2p0_rxvalid_rxactive	property p_rxvalid_with_rxactive; @(posedge utmi_clk) utmi_rxvalid |-> utmi_rxactive; endproperty
3	usb2p0_txvalid_txready	property p_txvalid_txready_handshake; @(posedge utmi_clk) utmi_txvalid |-> ##[0:5] utmi_txready; endproperty
4	usb2p0_txvalid_txdata	property p_txdata_not_unknown; @(posedge utmi_clk) utmi_txvalid |-> (!$isunknown(utmi_txdata)); endproperty

5	usb2p0_rxvalid_rxdata	property p_rxdata_not_unknown; @(posedge utmi_clk) utmi_rxvalid |-> (!$isunknown(utmi_rxdata)); endproperty
6	usb2p0_linestate	property p_linestate_known; @(posedge utmi_clk) !$isunknown(utmi_linestate); endproperty
7	usb2p0_valid_validh_wordif	property p_ls_fs_mode_check; @(posedge utmi_clk) (!utmi_txvalidh && !utmi_rxvalidh) |-> (utmi_word_if == 1'b0); endproperty
8	usb2p0_valid_validh_wordif	property p_hs_mode_check; @(posedge utmi_clk) (utmi_txvalidh || utmi_rxvalidh) |-> (utmi_word_if == 1'b1); endproperty
9	usb2p0_vbus	property p_vbus_valid; @(posedge utmi_clk) utmiotg_vbusvalid |-> utmisrp_bvalid; endproperty

10	usb2p0_dp_dm	property p_dp_dm_pulldown; @(posedge utmi_clk) !(utmiotg_dppulldown && utmiotg_dmpulldown); endproperty

Table No 1 Assertion

17.0 Coverage

SI.NO	Cover_group_name	cover_point_name	Bins

1	covergroup usb_pid_cg;	USB_PID_CP :coverpoint pidsample_host_tx	bit [DATA_WIDTH-1:0] pidsample_host_tx;
			
			USB_PID_CP : coverpoint pidsample_host_tx {
			bins setup_pid_htx = {8'hD2};
			bins data0_pid_htx = {8'h3C};
			bins ack_pid_htx = {8'h2D};
			bins in = {8'h96};
			bins out = {8'h1E};
			bins data1 = {8'hB4};
			}

2		ADDR: coverpoint usb_seq_item.addr	bit [7:0]ADDRESS;
			
			ADDR: coverpoint usb_seq_item.addr{
			bins device_addr0 = {0};
			ignore_bins others = {[0:127]} with (item != 0);
			}

3		ENDPOINT : coverpoint usb_seq_item.endp	bit [3:0] endp;
			
			ENDPOINT : coverpoint usb_seq_item.endp {
			bins control_ep = {0};
			bins bulk_in_ep = {1};
			bins bulk_out_ep = {2};
			bins interrupt_in_ep = {3};
			bins interrupt_out_ep = {5};
			bins iso_in_ep = {6};
			bins iso_out_ep = {7};
			ignore_bins others = {[8:15]};
			}

4		BREQUEST: coverpoint usb_seq_item.bRequest	BREQUEST: coverpoint usb_seq_item.bRequest {
			bins get_status = {8'h00};
			bins clear_feature = {8'h01};
			bins set_feature = {8'h03};
			bins set_address = {8'h05};
			bins get_descriptor = {8'h06};
			bins get_config = {8'h08};
			bins set_config = {8'h09};
			bins get_interface = {8'h0A};
			bins set_interface = {8'h0B};
			ignore_bins others = {[0:255]} with (!(item inside {
			8'h00,8'h01,8'h03,8'h05,8'h06,8'h07,8'h08,8'h09,8'h0A,8'h0B
			}));
			}

5		BMREQUESTTYPE : coverpoint usb_seq_item.bmRequestType	BMREQUESTTYPE : coverpoint usb_seq_item.bmRequestType {
			bins std_out_device = {8'h00};
			bins std_device = {8'h01};
			bins std_in_device = {8'h80};
			ignore_bins = {[0:255]} with (!(item inside {8'h00,8'h01,8'h80,8'h81}));
			}

6		WVALUE : coverpoint usb_seq_item.wValue	WVALUE : coverpoint usb_seq_item.wValue {
			bins zero = {16'h0000};
			bins one = {16'h0001};
			ignore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0001}));
			}

7		WINDEX : coverpoint usb_seq_item.wIndex	WINDEX : coverpoint usb_seq_item.wIndex {
			bins wzero = {16'h0000};
			bins wone = {16'h0001};
			bins val_81 = {16'h0081};
			inore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0001,16'h0081}));

8		WLENGTH : coverpoint usb_seq_item.wLength	WLENGTH : coverpoint usb_seq_item.wLength {
			bins zero = {16'h0000};
			bins two = {16'h0002};
			bins desc = {16'h0012};
			ignore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0002,16'h0012}));
			}

9		BLENGTH : coverpoint usb_seq_item.blength	BLENGTH : coverpoint usb_seq_item.blength {
			bins device_len = {8'h12};
			ignore_bins = {[0:255]} with (item != 8'h12);
			}

10		DESC_TYPE : coverpoint usb_seq_item.bdescriptors_type	DESC_TYPE : coverpoint usb_seq_item.bdescriptors_type {
			bins device_desc = {8'h01};
			ignore_bins others = {[0:255]} with (item != 8'h01);
			}

11		BCD_USB : coverpoint usb_seq_item.bcd_usb	BCD_USB : coverpoint usb_seq_item.bcd_usb {
			bins usb_1 = {16'h0110};
			ignore_bins others = {[0:65535]} with (item != 16'h0110);
			}

12		DEVICE_CLASS : coverpoint usb_seq_item.bDevice_class	DEVICE_CLASS : coverpoint usb_seq_item.bDevice_class {
			bins bdeviceclass = {8'h00};
			ignore_bins = {[0:255]} with (item != 8'h00);
			}

13		DEVICE_SUBCLASS : coverpoint usb_seq_item.bDevice_subclass	DEVICE_SUBCLASS : coverpoint usb_seq_item.bDevice_subclass {
			bins subclass = {8'h00};
			ignore_bins s = {[0:255]} with (item != 8'h00);
			}

14		DEVICE_PROTOCOL : coverpoint usb_seq_item.bDevice_protocol	DEVICE_PROTOCOL : coverpoint usb_seq_item.bDevice_protocol {
			bins protocol = {8'h00};
			ignore_bins= {[0:255]} with (item != 8'h00);

15		MAX_PACKET : coverpoint usb_seq_item.bMax_packetsize	MAX_PACKET : coverpoint usb_seq_item.bMax_packetsize {
			bins packetsize = {8'h40};
			ignore_binss = {[0:255]} with (item != 8'h40);
			}

16		VENDOR_ID : coverpoint usb_seq_item.idvendor	VENDOR_ID : coverpoint usb_seq_item.idvendor {
			bins vendor_781 = {16'h0781};
			ignore_bins = {[0:65535]} with (item != 16'h0781);
			}

17		PRODUCT_ID : coverpoint usb_seq_item.idproduct	PRODUCT_ID : coverpoint usb_seq_item.idproduct {
			bins product_5567 = {16'h5567};
			ignore_bins = {[0:65535]} with (item != 16'h5567);
			}

18		DEVICE_BCD : coverpoint usb_seq_item.bcdDevice	DEVICE_BCD : coverpoint usb_seq_item.bcdDevice {
			bins dev_ver = {16'h0126};
			ignore_bins = {[0:65535]} with (item != 16'h0126);
			}

19		MANUF_STR : coverpoint usb_seq_item.imanufacture	MANUF_STR : coverpoint usb_seq_item.imanufacture {
			bins mfg = {8'h01};
			ignore_bins = {[0:255]} with (item != 8'h01);
			}

20		PROD_STR : coverpoint usb_seq_item.iproduct	PROD_STR : coverpoint usb_seq_item.iproduct {
			bins product = {8'h02};
			ignore_bins = {[0:255]} with (item != 8'h02);
			}

21		SERIAL_STR : coverpoint usb_seq_item.iserial_number	SERIAL_STR : coverpoint usb_seq_item.iserial_number {
			bins serial = {8'h03};
			ignore_bins = {[0:255]} with (item != 8'h03);
			}

22		NUM_CONFIG : coverpoint usb_seq_item.bNum_configuration	NUM_CONFIG : coverpoint usb_seq_item.bNum_configuration {
			bins cfg = {8'h01};
			ignore_bins others = {[0:255]} with (item != 8'h01);
			}

23		TX_VALID : coverpoint usb_seq_item.tx_valid;	TX_VALID : coverpoint usb_seq_item.tx_valid;
24		TX_VALIDH : coverpoint usb_seq_item.tx_validh;	TX_VALIDH : coverpoint usb_seq_item.tx_validh;
25		RX_VALID : coverpoint usb_seq_item.rx_valid;	RX_VALID : coverpoint usb_seq_item.rx_valid;
26		RX_VALIDH : coverpoint usb_seq_item.rx_validh;	RX_VALIDH : coverpoint usb_seq_item.rx_validh;

27		WORD_IF : coverpoint usb_seq_item.word_if;	WORD_IF : coverpoint usb_seq_item.word_if;

28		OP_MODE : coverpoint usb_seq_item.op_mode	OP_MODE : coverpoint usb_seq_item.op_mode {
			bins hs_only = {2'b00};
			ignore_bins others = {[0:3]} with (item != 2'b00);
			}

29		TERM_SELECT : coverpoint usb_seq_item.term_select;	TERM_SELECT : coverpoint usb_seq_item.term_select;
30		XCVR_SELECT : coverpoint usb_seq_item.xcvr_select;	XCVR_SELECT : coverpoint usb_seq_item.xcvr_select;
31		SUSPEND_N : coverpoint usb_seq_item.suspend_n;	SUSPEND_N : coverpoint usb_seq_item.suspend_n;
32		FSLS_SERIALMODE : coverpoint usb_seq_item.fsls_serialmode;	FSLS_SERIALMODE : coverpoint usb_seq_item.fsls_serialmode;
33		FSLS_LOW_POWER : coverpoint usb_seq_item.fsls_low_power;	FSLS_LOW_POWER : coverpoint usb_seq_item.fsls_low_power;
34		OTG_DPPULLDOWN : coverpoint usb_seq_item.otg_dppulldown;	OTG_DPPULLDOWN : coverpoint usb_seq_item.otg_dppulldown;
35		OTG_DMPULLDOWN : coverpoint usb_seq_item.otg_dmpulldown;	OTG_DMPULLDOWN : coverpoint usb_seq_item.otg_dmpulldown;
36		UTMI_VALID_CROSS : cross TX_VALID, RX_VALID;	UTMI_VALID_CROSS : cross TX_VALID, RX_VALID;

Table No 2 Coverage Table

18.0 Testcases Table:

S.No	Testcase name	ETA	Status
1	usb2p0_pm_power_connect	11/19/2025	Pass
2	usb2p0_pm_power_disconnect	11/19/2025	Pass
3	usb2p0_ls_speed_detect	11/20/2025	Pass
4	usb2p0_fs_speed_detect	11/20/2025	Pass
5	usb2p0_hs_speed_detect	11/20/2025	Pass
6	USB2p0_LS_CT_clear_feature_test	11/24/2025	Pass
7	USB2p0_FS_CT_clear_feature_test	11/24/2025	Pass
8	USB2p0_HS_CT_clear_feature_test	1/22/2026	Pass
9	USB2p0_LS_CT_set_address_test	11/26/2025	Pass
10	USB2p0_FS_CT_set_address_test	11/26/2025	Pass
11	USB2p0_HS_CT_set_address_test	1/27/2026	Pass
12	USB2p0_LS_CT_set_configuration_test	11/28/2025	Pass
13	USB2p0_FS_CT_set_configuration_test	11/28/2025	Pass
14	USB2p0_HS_CT_set_configuration_test	1/30/2026	Pass
15	USB2p0_LS_CT_set_feature_test	12/2/2025	Pass
16	USB2p0_FS_CT_set_feature_test	12/2/2025	Pass
17	USB2p0_HS_CT_set_feature_test	2/4/2026	Pass
18	USB2p0_LS_CT_set_interface_test	12/5/2025	Pass
19	USB2p0_FS_CT_set_interface_test	12/5/2025	Pass
20	USB2p0_HS_CT_set_interface_test	2/9/2026	Pass
21	USB2p0_LS_CT_get_descriptor_test	12/8/2025	Pass
22	USB2p0_FS_CT_get_descriptor_test	12/8/2025	Pass
23	USB2p0_HS_CT_get_descriptor_test	2/13/2026	Pass
24	usb2p0_CT_LS_get_status_test	12/12/2025	Pass
25	usb2p0_CT_FS_get_status_test	12/12/2025	Pass
26	usb2p0_CT_HS_get_status_test	2/18/2026	Pass
27	usb2p0_CT_LS_get_config_test	12/16/2025	Pass
28	usb2p0_CT_FS_get_config_test	12/16/2025	Pass
29	usb2p0_CT_HS_get_config_test	2/23/2026	Pass
30	usb2p0_CT_LS_get_interface_test	12/19/2025	Pass
31	usb2p0_CT_FS_get_interface_test	12/19/2025	Pass
32	usb2p0_CT_HS_get_interface_test	2/26/2026	Pass
33	usb2p0_LS_INTR_IN_transfer	12/24/2025	Pass
34	usb2p0_FS_INTR_IN_transfer	12/24/2025	Pass
35	usb2p0_HS_INTR_IN_transfer	3/2/2026	Pass
36	usb2p0_LS_INTR_OUT_transfer	1/7/2026	Pass
37	usb2p0_FS_INTR_OUT_transfer	1/7/2026	Pass

38	usb2p0_HS_INTR_OUT_transfer	3/5/2026	Pass
39	usb2p0_FS_BULK_IN_transfer	1/12/2026	Pass
40	usb2p0_HS_BULK_IN_transfer	3/9/2026	Pass
41	usb2p0_FS_BULK_OUT_transfer	1/12/2026	Pass
42	usb2p0_HS_BULK_OUT_transfer	3/12/2026	Pass
43	usb2p0_FS_ISO_IN_transfer	1/16/2026	Pass
44	usb2p0_HS_ISO_IN_transfer	3/16/2026	Pass
45	usb2p0_FS_ISO_OUT_transfer	1/16/2026	pass
46	usb2p0_HS_ISO_OUT_transfer	3/20/2026	Pass
47	USB2p0_LS_CT_clear_feature_error_test	3/23/2026	Pass
48	USB2p0_FS_CT_clear_feature_error_test	3/23/2026	Pass
49	USB2p0_HS_CT_clear_feature_error_test	5/13/2026	Pass
50	USB2p0_LS_CT_set_address_error_test	3/27/2026	Pass
51	USB2p0_FS_CT_set_address_error_test	3/27/2026	Pass
52	USB2p0_HS_CT_set_address_error_test	5/13/2026	Pass
53	USB2p0_LS_CT_set_configuration_error_test	3/31/2026	Pass
54	USB2p0_FS_CT_set_configuration_error_test	4/1/2026	Pass
55	USB2p0_HS_CT_set_configuration_error_test	5/13/2026	Pass
56	USB2p0_LS_CT_set_feature_error_test	4/6/2026	Pass
57	USB2p0_FS_CT_set_feature_error_test	4/9/2026	Pass
58	USB2p0_HS_CT_set_feature_error_test	5/15/2026	Pass
59	USB2p0_LS_CT_set_interface_error_test	4/13/2026	Pass
60	USB2p0_FS_CT_set_interface_error_test	4/13/2026	Pass
61	USB2p0_HS_CT_set_interface_error_test	5/15/2026	Pass
62	USB2p0_LS_CT_get_descriptor_error_test	4/16/2026	Pass
63	USB2p0_FS_CT_get_descriptor_error_test	4/16/2026	Pass
64	USB2p0_HS_CT_get_descriptor_error_test	5/19/2026	Pass
65	usb2p0_CT_LS_get_status_error_test	4/20/2026	Pass
66	usb2p0_CT_FS_get_status_error_test	4/20/2026	Pass
67	usb2p0_CT_HS_get_status_error_test	5/19/2026	Pass
68	usb2p0_CT_LS_get_config_error_test	4/23/2026	Pass
69	usb2p0_CT_FS_get_config_error_test	4/23/2026	Pass
70	usb2p0_CT_HS_get_config_error_test	5/19/2026	Pass
71	usb2p0_CT_LS_get_interface_error_test	4/27/2026	Pass
72	usb2p0_CT_FS_get_interface_error_test	4/27/2026	Pass
73	usb2p0_CT_HS_get_interface_error_test	5/19/2026	Pass
74	usb2p0_LS_INTR_IN_error_transfer	4/29/2026	Pass
75	usb2p0_FS_INTR_IN_error_transfer	4/29/2026	Pass
76	usb2p0_HS_INTR_IN_error_transfer	5/20/2026	Pass
77	usb2p0_LS_INTR_OUT_transfer	5/5/2026	Pass
78	usb2p0_FS_INTR_OUT_transfer	5/5/2026	Pass

79	usb2p0_HS_INTR_OUT_transfer	5/20/2026	Pass
80	usb2p0_FS_BULK_IN_error_transfer	5/7/2026	Pass
81	usb2p0_HS_BULK_IN_error_transfer	5/21/2026	Pass
82	usb2p0_FS_BULK_OUT_error_transfer	5/7/2026	Pass
83	usb2p0_HS_BULK_OUT_error_transfer	5/21/2026	Pass
84	usb2p0_FS_ISO_IN_error_transfer	5/11/2026	Pass
85	usb2p0_HS_ISO_IN_error_transfer	5/22/2026	Pass
86	usb2p0_FS_ISO_OUT_error_transfer	5/11/2026	Pass
87	usb2p0_HS_ISO_OUT_error_transfer	5/22/2026	Pass
88	usb2p0_LS_crc5_error_test	5/19/2026	pass
89	usb2p0_FS_crc5_error_test	5/20/2026	pass
90	usb2p0_HS_crc5_error_test	5/21/2026	pass
91	usb2p0_LS_crc16_error_test	5/19/2026	pass
92	usb2p0_FS_crc16_error_test	5/20/2026	pass
93	usb2p0_HS_crc16_error_test	5/21/2026	pass
94	usb2p0_LS_CT_random_testcase	5/21/2026	Pass
95	usb2p0_FS_CT_random_testcase	5/21/2026	Pass
96	usb2p0_HS_CT_random_testcase	5/21/2026	Pass
97	Usb2p0_LS _interrupt_in_ random _testcase	5/22/2026	Pass
98	Usb2p0_FS _interrupt_in_ random _testcase	5/22/2026	Pass
99	Usb2p0_HS _interrupt_in_ random _testcase	5/22/2026	Pass
100	Usb2p0_LS _interrupt_out_ random _testcase	5/22/2026	Pass
101	Usb2p0_FS _interrupt_out_ random _testcase	5/22/2026	Pass
102	Usb2p0_HS _interrupt_out_ random _testcase	5/22/2026	Pass
103	usb2p0_FS_bulk_in_random_testcase	5/25/2026	Pass
104	usb2p0_HS_bulk_in_random_testcase	5/25/2026	Pass
105	usb2p0_FS_bulk_out_random_testcase	5/25/2026	Pass
106	usb2p0_HS_bulk_out_random_testcase	5/25/2026	Pass
107	usb2p0_FS_iso_in_random_testcase	5/26/2026	Pass
108	usb2p0_HS_iso_in_random_testcase	5/26/2026	Pass
109	usb2p0_FS_iso_out_random_testcase	5/26/2026	Pass
110	usb2p0_HS_iso_out_random_testcase	5/26/2026	Pass

Table No 3 Testcase Table